COMP4153 Natural Language Processing

Information Extraction and Structured NLP

Chin Poo Lee
School of Computer Science
University of Nottingham Ningbo China

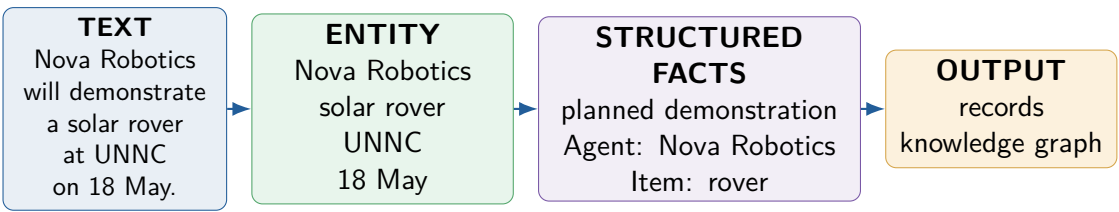
Learning outcomes

By the end of this lecture, you should be able to:

- formulate entity extraction as BIO tagging or span extraction;
- explain the difference between entity recognition, entity typing and entity linking;
- explain ordered entity pairs and ways to infer relations from text;
- represent event mentions using event types and role-filled arguments;
- construct structured records from extracted entities, relations and events;
- identify common extraction errors and choose suitable evaluation units.

From text to structured information

Information Extraction (IE). IE converts information expressed in text into structured facts that can be searched, queried and combined.



- **Entity extraction** identifies relevant entity mentions and their types.
- **Relation and event extraction** connects mentions and describes events.
- **Record construction** organises the results using an application schema.

Example

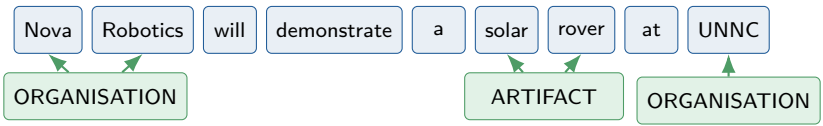
> Nova Robotics, a student team from UNNC, will demonstrate a solar rover at the Ningbo Campus Innovation Showcase on 18 May. The project lead, Lin Mei, said the rover uses a lightweight vision model to follow marked paths.

From this passage, we may extract:

- **Entity mentions**: Nova Robotics (ORG), UNNC (ORG), solar rover (ARTIFACT), 18 May (DATE), Lin Mei (PERSON).
- **Possible relation**: (Nova Robotics, associated_with, UNNC).
- **Planned event**: demonstration; Agent = Nova Robotics; Item = rover; Date = 18 May; Occasion = Ningbo Campus Innovation Showcase.

Entity extraction: find the span and its type

Entity mention. An entity mention is a span of text that refers to an object, concept, organisation, person, place, time or domain-specific item.



Every entity extractor must answer two questions:

1. **Boundary:** which tokens belong to the entity?
2. **Type:** what kind of entity is it?

$$\text{Entity extraction} = \text{boundary detection} + \text{type classification}.$$

NER is one type of entity extraction

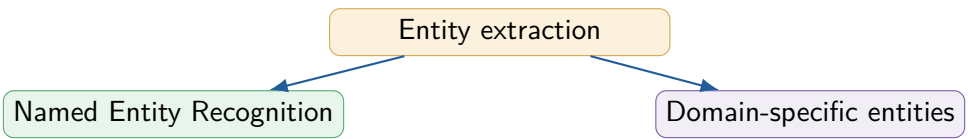
Named Entity Recognition (NER). NER identifies mentions of named entities and assigns predefined types such as PERSON, ORGANISATION, LOCATION or DATE.

Traditional NER

- Lin Mei → PERSON
- UNNC → ORGANISATION
- Ningbo → LOCATION
- 18 May → DATE

Domain-specific entity extraction

- solar rover → ARTIFACT
- lightweight vision model → METHOD
- marked paths → ENVIRONMENT FEATURE



Two common ways to formulate entity extraction

The same entity-extraction task can be represented in different ways.
Example: Nova Robotics will demonstrate solar rover...

1. Token-based tagging
Assign one label to each token:

$$(x_1, \ldots, x_n) \to (y_1, \ldots, y_n).$$

- x_k: input token at position k.
- y_k: label assigned to token x_k.

$$(y_1, y_2, y_3) = (\text{B-ORG}, \text{I-ORG}, \text{O}),$$
$$(y_4, y_5, y_6) = (\text{O}, \text{B-ART}, \text{I-ART}).$$

2. Span-based extraction
Predict each entity directly as:

$$(i, j, t).$$

- i, j: inclusive start and end token positions.
- t: entity type.

$$(1, 2, \text{ORG}), \quad (5, 6, \text{ARTIFACT}).$$

Token-based formulation: BIO tagging

BIO tagging. BIO tags encode entity boundaries and types: B for beginning, I for inside, and O for outside.

Token	BIO label	Interpretation
Nova	B-ORG	start of organisation
Robotics	I-ORG	same organisation continues
will	O	outside an entity
demonstrate	O	outside an entity
solar	B-ART	start of artifact
rover	I-ART	same artifact continues

$$\text{B-ORG}, \text{I-ORG} \quad \Rightarrow \quad [\text{Nova Robotics}]_{\text{ORG}}.$$

- **B**: beginning of an entity mention.
- **I**: continuation of the same entity mention.
- **O**: outside a recognised entity mention.

Why do we need B and I?

Suppose two organisations occur next to each other.



If we only used the label ORG,

$$\text{ORG}, \text{ORG}, O, O, O,$$

it would be unclear whether "Google Microsoft" is one entity or two.

BIO removes the ambiguity:

$$\text{B-ORG}, \text{B-ORG}, O, O, O.$$

This means:

$$[\text{Google}]_{\text{ORG}} \qquad [\text{Microsoft}]_{\text{ORG}}.$$

How does a system predict BIO labels?

A BIO tagger maps an input token sequence to a valid sequence of boundary-and-type labels.



The predicted sequence is

$$\hat{\boldsymbol{y}} = F_{\mathrm{BIO}}(x), \qquad \hat{y}_i \in \mathcal{Y}_{\mathrm{BIO}}.$$

- $x = (x_1, \ldots, x_n)$: input tokens.
- F_{BIO}: token-sequence labelling procedure.
- $\hat{\boldsymbol{y}} = (\hat{y}_1, \ldots, \hat{y}_n)$: predicted tags.
- $\mathcal{Y}_{\mathrm{BIO}}$: allowed BIO labels.

For example, the phrase “Nova Robotics” receives the tags

B-ORG, I-ORG.

Limitation of token tagging

A **single, flat BIO sequence** assigns one label to each token.

Consider the mention:

University of Nottingham Ningbo China.

Suppose the annotation schema requires both:

$$[\text{University of Nottingham Ningbo China}]_{\text{ORG}}$$

and

$$[\text{Ningbo}]_{\text{LOCATION}}.$$

The word "Ningbo" now belongs to two nested mentions.

one BIO layer: flat mentions → multiple spans / layers: nesting

Span-based extraction: predict the entity directly

Span-based extraction. Represent each entity mention by its token boundaries and type.

$$\text{mention} = (i, j, t), \qquad 1 \le i \le j \le n.$$

- i, j: inclusive start and end token positions.
- t: entity type.
- n: number of tokens.

Example token sequence:

Index	1	2	3	4	5	6
Token	Nova	Robotics	built	a	solar	rover

Predicted mentions:

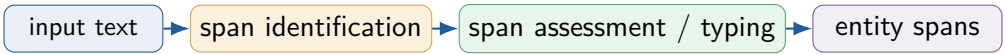
$$(1, 2, \text{ORG}), \qquad (5, 6, \text{ARTIFACT}).$$

Each output identifies both the **boundary** and the **entity type**:

$$\underbrace{(i, j)}_{\text{boundary}} \quad + \quad \underbrace{t}_{\text{type}}.$$

How does a span model work?

In a span-based formulation, the system identifies text spans and assigns entity types.



For a candidate-based span extractor:

$$\hat{t}_{ij} = F_{\text{span}}(x, i, j), \qquad \hat{t}_{ij} \in \mathcal{T} \cup \{\text{NONE}\}.$$

- x: input text.
- (i, j): candidate span boundaries.
- F_{span}: span assessment procedure.
- $\hat{t}_{ij}$: predicted label.
- $\mathcal{T}$: entity types.
- NONE: candidate is not an entity mention.

Other systems may predict or generate spans directly.

Worked example: candidate spans

Sentence:

Index	1	2	3	4	5	6	7
Token	Lin	Mei	leads	Nova	Robotics	in	Ningbo

For an illustrative candidate-based implementation restricted to spans of at most three tokens:

Candidate	Text	Illustrative label
$(1, 1)$	Lin	NONE
$(1, 2)$	Lin Mei	PERSON
$(4, 4)$	Nova	NONE
$(4, 5)$	Nova Robotics	ORG
$(7, 7)$	Ningbo	LOCATION

The task is to identify valid span boundaries and assign the correct type. Non-entity candidates are rejected.

Token tagging or span extraction?

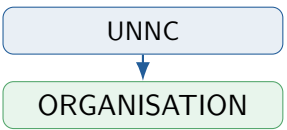
	Token tagging / BIO	Span-based extraction
Output format	one tag per token	(i, j, t) per mention
Flat entities	directly supported	directly supported
Nested entities	require additional tagging layers or schemes	can represent overlapping spans
Typical computation	sequence-level labelling	may score many candidate spans
Relations downstream	recover mention spans from tags	mention spans already explicit

Both describe **what to extract**, not which neural architecture must be used.

Entity extraction tells us what; entity linking tells us which

Entity linking. Entity linking maps an extracted mention to a canonical entity in a knowledge base, database or catalogue.

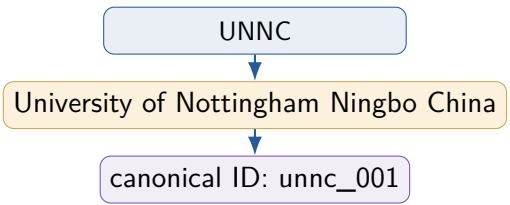
Entity extraction



Answers:

"What is this mention?"

Entity linking



Answers:

"Which entity is it?"

Why is entity linking useful?

Different surface forms may refer to the same real-world entity.

$$\text{UNNC} \quad \equiv \quad \text{University of Nottingham Ningbo China.}$$

Without linking, a database may treat them as two separate objects.

With linking:

$$\text{UNNC} \rightarrow \texttt{unnc_001}$$

and

$$\text{University of Nottingham Ningbo China} \rightarrow \texttt{unnc_001}.$$

Now information from different documents can be combined reliably.

- search can retrieve all mentions of the same entity;
- relations can be merged across documents;
- knowledge graphs can use stable node identifiers.

From entities to relations

Relation extraction. Identify a semantic relation between two entity mentions (or linked entities), as supported by text and the chosen schema.

Example facts from short sentences such as "Lin Mei leads Nova Robotics" and "Nova Robotics built a solar rover":

Head entity	Relation	Tail entity
Lin Mei	leads	Nova Robotics
Nova Robotics	built	solar rover

Notation:

$$\underbrace{(e_1, r, e_2)}_{\text{relation triple}} \quad \text{versus} \quad \underbrace{(e_1, e_2)}_{\text{ordered candidate pair}} .$$

- e_1: head (first entity).
- r: relation label.
- e_2: tail (second entity).
- order matters for directed relations.

Predicting relations for ordered entity pairs

After identifying entity mentions, a system can examine **ordered candidate pairs**. Suppose the mentions are:

- e_1: Lin Mei.
- e_2: Nova Robotics.
- e_3: solar rover.

Candidate pairs include (e_1, e_2), (e_1, e_3) and (e_2, e_3). For each pair, determine which relation is supported by the text and the schema:

$$(\text{Lin Mei}, \text{Nova Robotics}) \longrightarrow (\text{Lin Mei}, \text{leads}, \text{Nova Robotics}).$$

$$(\text{Lin Mei}, \text{solar rover}) \longrightarrow \texttt{no_relation} \quad \text{(for the given schema and text)}.$$

Relations depend on context, not only entity types

The same entity types can participate in different relations.

Sentence A

Lin Mei **leads** Nova Robotics.

(Lin Mei, leads, Nova Robotics)

Sentence B

Lin Mei **visited** Nova Robotics.

(Lin Mei, visited, Nova Robotics)

Both involve a PERSON and an ORG, but the evidence and relation differ.

- Consider both entity mentions and their surrounding context.
- Consider the allowed relation types and their argument roles.
- Preserve the direction of any directed relation.

Relation direction also matters

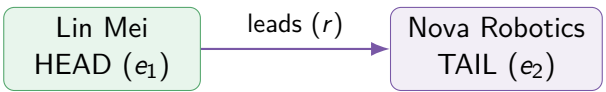
For a directed relation, swapping the entities generally changes the meaning.

Supported by the sentence

(Lin Mei, leads, Nova Robotics)

Not supported

(Nova Robotics, leads, Lin Mei)



ordered pair: (e_1, e_2) extracted triple: (e_1, r, e_2).

Specifying the target entity pair

Target entity pair. The selected head and tail mentions for which a relation is to be extracted. A sentence may mention several entities. The extraction procedure must distinguish which mentions play the head and tail roles.

```
Sentence (with optional entity markers):
[HEAD: PERSON] Lin Mei [/HEAD] leads
[TAIL: ORG] Nova Robotics [/TAIL],
which built a solar rover for the campus showcase.

Question: What relation connects HEAD to TAIL?
Allowed labels: leads, built, no_relation
```

$$(e_1, e_2) = (\text{Lin Mei}, \text{Nova Robotics}) \quad \Rightarrow \quad (e_1, \text{leads}, e_2).$$

Representing a pair for relation extraction

Pair representation. A representation of the selected head and tail entities and the evidence needed to assess their relation.



Conceptually:

$$\hat{\mathcal{R}}_{12} = F_{\text{rel}}(x, e_1, e_2), \qquad \hat{\mathcal{R}}_{12} \subseteq \mathcal{R}.$$

- x: source text.
- (e_1, e_2): ordered entity pair.
- F_{rel}: relation-inference procedure.
- $\mathcal{R}$: allowed relation labels.
- $\hat{\mathcal{R}}_{12}$: predicted relation-label set (possibly empty).

How is the relation label predicted?

Given the text and ordered pair (e_1, e_2), identify every relation supported by the evidence and the schema.

$$(e_1, e_2) = (\text{Lin Mei}, \text{Nova Robotics}).$$

For "Lin Mei leads Nova Robotics", consider these possible labels:

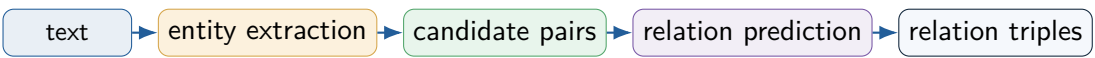
Candidate relation	Evidence in sentence	Extract?
leads	explicitly stated	yes
built	not stated for this pair	no
visited	not stated	no

The resulting triple is

$$(\text{Lin Mei}, \text{leads}, \text{Nova Robotics}).$$

Pipeline relation extraction

Pipeline extraction. A modular process in which entity mentions are identified before their relations are inferred.



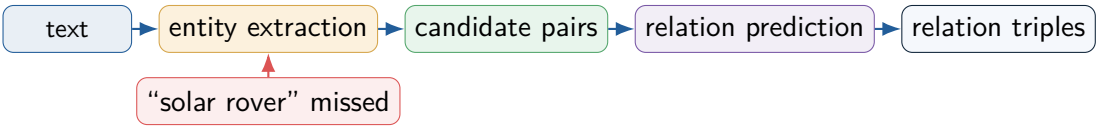
Example:

$$\text{“Lin Mei leads Nova Robotics”} \rightarrow \{\text{Lin Mei}, \text{Nova Robotics}\}$$

$$\rightarrow (\text{Lin Mei}, \text{Nova Robotics}) \rightarrow (\text{Lin Mei}, \text{leads}, \text{Nova Robotics}).$$

Pipeline systems can propagate errors

A pipeline can only consider relation candidates built from the detected entity mentions.



If “solar rover” is missed, the candidate pair

$$(\text{Nova Robotics}, \text{solar rover})$$

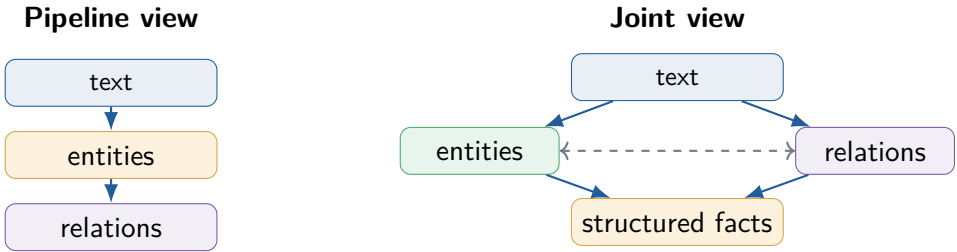
may not be considered, and the supported triple

$$(\text{Nova Robotics}, \text{built}, \text{solar rover})$$

may be missed as a consequence.

Joint extraction predicts entities and relations together

Joint extraction. Identify entities and their relations through a shared or coordinated prediction process.



Entity and relation evidence can inform each other. For example,

$$(\text{Lin Mei}, \text{leads}, \text{Nova Robotics})$$

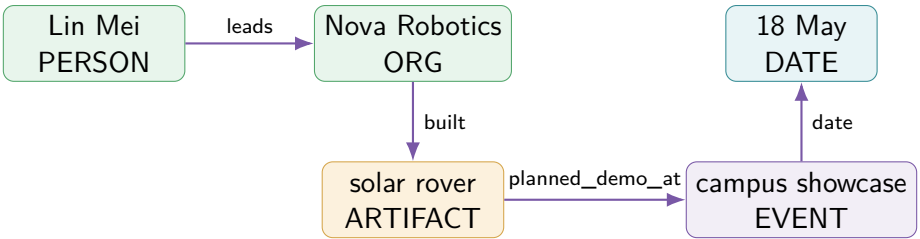
helps support the entity roles PERSON and ORG in the given schema.

Pipeline or joint extraction?

	Pipeline extraction	Joint extraction
Organisation	separate extraction stages	shared or coordinated decisions
Modularity	often higher	often lower
Training / debugging	individual stages easier to inspect	interactions may be harder to analyse
Error propagation	upstream entity errors affect candidates	shared reasoning may mitigate some errors
Information sharing	limited unless designed in	entity and relation evidence can interact

Relation extraction builds a graph

A set of supported relation triples can be represented as a directed, labelled graph.



Why are extracted relations useful?

Once the text has been converted into a graph, software can follow relations to answer structured questions.

Question:

Who leads the team that built the solar rover?

Reasoning:

$$\text{solar rover} \xleftarrow{\text{built}} \text{Nova Robotics} \xleftarrow{\text{leads}} \text{Lin Mei}.$$

Therefore,

$$\boxed{\text{Lin Mei}}$$

From relations to events

Event extraction. Identify events in text and their role-filled arguments, including participants, items and dates.

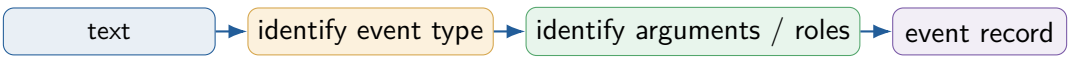
Unlike a relation such as (Nova Robotics, built, solar rover), an event can group several arguments around one planned or completed occurrence.

> Nova Robotics will **demonstrate** a solar rover at the Ningbo Campus Innovation Showcase on 18 May.

Event element	Extracted value
Event type / trigger	Demonstration / demonstrate
Status	planned (future)
Agent	Nova Robotics
Artifact	solar rover
Occasion	Ningbo Campus Innovation Showcase
Date	18 May

Event extraction has two linked tasks

Two common, complementary subtasks are **event identification** and **argument extraction**.



1. Event identification

- detect an event mention;
- assign its event type;
- optionally locate a trigger.

  demonstrate → Demonstration.

2. Argument extraction

- locate the supporting entity or text span;
- assign its semantic role in the event.

  Nova Robotics → Agent.

Event trigger. A word or phrase that signals an event mention under an event-annotation scheme.

Sentence fragment	Possible trigger	Event type
Lin Mei **announced** the plan	announced	Announcement
Nova Robotics will **demonstrate** the rover	demonstrate	Demonstration
The team **launched** the system	launched	Launch

$$\text{event mention} \longrightarrow \text{event type}$$

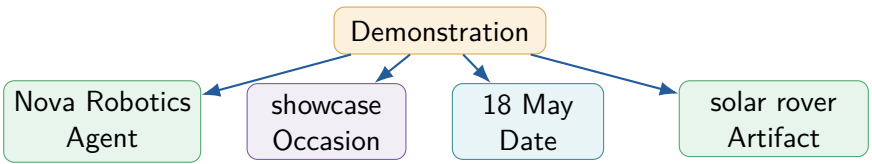
Not every verb signals a target event; the valid event types and annotation rules are task-specific.

Argument extraction: who or what participated?

For an identified event, find the supporting argument spans and their semantic roles.

Nova Robotics will demonstrate a solar rover at the showcase on 18 May.

$$\text{Agent} = \text{Nova Robotics}, \quad \text{Artifact} = \text{solar rover},$$
$$\text{Occasion} = \text{showcase}, \quad \text{Date} = 18\ \text{May}.$$



A sentence may contain multiple events

An extractor must keep different event mentions and their arguments separate.

> Lin Mei **announced** that Nova Robotics will **demonstrate** its solar rover in the student innovation area on 18 May.

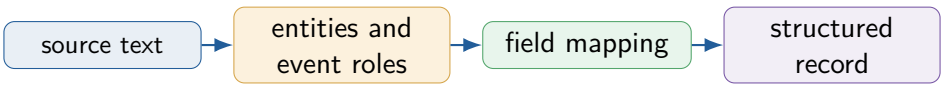
Field	Announcement event	Demonstration event
Trigger	announced	demonstrate
Agent / speaker	Lin Mei	Nova Robotics
Content / artifact	plan to demonstrate	solar rover
Location	not specified	student innovation area
Date	not specified	18 May
Status	reported occurrence	planned (future)

Arguments must be attached to the appropriate event rather than to every event in the sentence.

From event extraction to structured records

Record extraction. Map information identified in text into named fields required by an application.

Example: Nova Robotics will demonstrate its rover at UNNC on 18 May.



- **Entity extraction** identifies Nova Robotics, rover, UNNC and 18 May.
- **Event extraction** identifies the demonstration and its participants.
- **Record extraction** places these facts into application-specific fields.

Designing an extraction schema

Extraction schema. A specification of the fields, data types and constraints expected in the output.

Field	Type	Rule
`organisation`	string	required
`activity`	string	required; predefined activity types
`item`	string or null	optional
`location`	string or null	optional
`date`	string or null	optional; do not assume the year

- A schema determines **which facts** the application needs.
- Data types and allowed values support **automatic checks**.
- Missing values should follow an explicit policy, rather than be guessed.

Constructing a JSON record from extracted facts

Source: Nova Robotics will demonstrate its rover at UNNC on 18 May.

Evidence and field mapping

Field	Evidence
organisation	Nova Robotics
activity	demonstrate
item	rover
location	UNNC
date	18 May

Machine-readable output

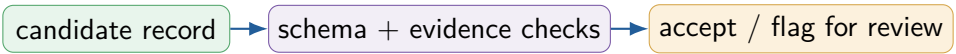
```
{
  "organisation": "Nova Robotics",
  "activity": "demonstration",
  "item": "rover",
  "location": "UNNC",
  "date": "18 May"
}
```

The activity label is normalised from the trigger; other fields retain their supported source values.

Validating extracted records

Record validation. Check that the populated fields conform to the schema and are supported by the source.

Check	Example
Structural validity	Required keys are present; types match the schema.
Missing values	If no date is stated, store `null`, not a guessed date.
Evidence	"UNNC" must be supported by the input text.
Ambiguity	"18 May" lacks a year; do not silently invent one.
Consistency	Conflicting locations should be flagged for review.



Evaluation starts with the output structure

Different extracted structures require different matching rules and evaluation units.



Task	Output unit	Example
Entities	mention span + type	(Lin Mei, PERSON)
Relations	head, relation, tail	(Lin Mei, leads, Nova Robotics)
Events	type + argument roles	Demonstration(Agent=Nova, Item=rover)
Fields	field-value pair	date = 18 May
Full records	schema-valid record	grounded JSON object

What counts as a correct prediction?

A prediction is correct only if it satisfies the task's chosen matching rule.

Entity extraction

Gold: (solar rover, ARTIFACT)

correct span + correct type

A prediction using only "rover" has an incorrect boundary.

Relation extraction

Gold: (Lin Mei, leads, Nova Robotics)

head + relation + tail

Swapping head and tail is incorrect for `leads`.

For events, compare event types and the correct argument-role assignments; for records, compare required field values and schema validity.

Precision, recall and F1 measure extraction quality

For a chosen evaluation unit:

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}, \qquad F_1 = \frac{2PR}{P + R}.$$

- TP, FP, FN: true positives, false positives, false negatives.
- P, R, F_1: precision, recall, F1 score.

Suppose an entity extractor produces the following under exact span-and-type matching:

	Predicted	Correct	Gold
Entity mentions	10	8	12

Then

$$P = \frac{8}{10} = 0.80, \qquad R = \frac{8}{12} \approx 0.67, \qquad F_1 = \frac{16}{22} \approx 0.73.$$

Precision and recall reflect different failure patterns

Low precision

The system predicts too many incorrect items.

many false positives

Example:

- labels non-entities as entities;
- predicts relations where none exist.

Improving precision usually means being more selective.

Low recall

The system misses many correct items.

many false negatives

Example:

- misses rare entity names;
- misses relations expressed indirectly.

Improving recall usually means finding more candidates.

F_1 balances precision and recall.

Exact match can be strict but useful

Gold entity:

"solar rover" → ARTIFACT

Prediction	Exact match?	Reason
solar rover → ARTIFACT	yes	boundary and type correct
solar → ARTIFACT	no	span too narrow
rover → ARTIFACT	no	span too narrow
solar rover → PRODUCT	no	type mismatch

Exact match is common because downstream applications may require precise spans and fields.

For diagnosis, however, partial-overlap measures can still help reveal **how** the prediction was wrong.

Common extraction errors occur at different stages

Error type	Example
Boundary error	extracts “rover” rather than “solar rover”
Type error	tags “Innovation Showcase” as LOCATION, not EVENT
Linking error	links UNNC to the wrong institution
Relation label error	predicts `visited` rather than `leads`
Relation direction error	predicts (Nova Robotics, leads, Lin Mei)
Missing event argument	omits the stated date “18 May”
Unsupported field	fabricates a contact person absent from the text
Schema error	uses `team` when the schema requires `organisation`

The appropriate correction depends on whether the error concerns mention boundaries, evidence, labels, roles or output constraints.

Error analysis connects failures to possible fixes

Observed problem	What to investigate
Many boundary errors	annotation rules, tokenisation, span representation
Many type errors	label definitions, training examples, class ambiguity
Low recall	candidate generation, rare examples, confidence threshold
Many no-relation mistakes	pair selection, negative examples, contextual representation
Unsupported field values	source spans, missing-value policy, evidence checking
Frequent schema failures	field mapping, data types, validator
Errors on ambiguous cases	additional context, knowledge base or human review

measure $\rightarrow$ categorize errors $\rightarrow$ identify cause $\rightarrow$ improve system

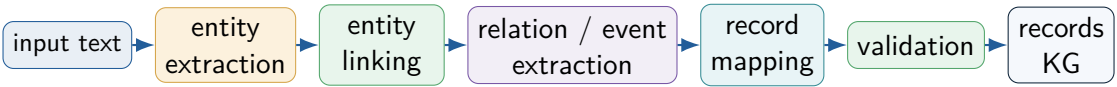
Evaluation should match the downstream goal

A single overall F1 score may not capture what matters to the application.

Application priority	Important evaluation question
Search / retrieval	Are important entities and relations being missed?
Database population	Are field values exact and schema-valid?
Knowledge graph	Are entities correctly linked and relation directions correct?
Human review tool	Are predictions easy to verify and accompanied by evidence?
Automated downstream system	Are unsupported or malformed outputs prevented?

End-to-end structured extraction system

One **possible modular workflow** is:



Different functions contribute different kinds of structure:

1. **Entity extraction**: identify mentions and types.
2. **Entity linking**: resolve mentions to canonical identities when needed.
3. **Relation / event extraction**: infer supported connections and event roles.
4. **Record construction**: map facts into the required fields.
5. **Validation**: check the schema and textual evidence before storage.